Politecnico di Milano Ingegneria del Software – a.a. 2004/05

Appello del 14 luglio 2005

Cognome
Nome
Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.). Non è possibile lasciare l'aula conservando il tema della prova in corso.
3. Tempo a disposizione: 2h., Punteggio totale a disposizione: 27/30.

Esercizio 1 (punti 10)

Il metodo statico *permutazione* riceve in ingresso un array *num* e restituisce un booleano. Il valore restituito vale true se, e solo se, l'array num contiene una permutazione dei valori 0, 1, ..., num.length-1.
Ad esempio, per gli array [0 3 4 2 1] e [1 3 2] il metodo restituisce true, mentre per [0 1 2 3 5] o [1 2 2 3 4 5] il metodo restituisce false..

a) Si scriva la specifica formale in JML del metodo, senza utlizzare commenti.

```
//@ requires num != null;
.....................................................................
//@ ensures
//@ \result ==(\forall int i; 0<= i && i <num.length;
//@                    (\exists int j; 0<=j && j<num.length; num[j] == i));

public static boolean permutazione(int [] num)
```

b) Si elenchi il minimo numero di casi di test funzionali per il metodo che ritenete sufficientemente adeguati per il suo test, specificando sia i dati che il valore ritornato atteso.
(utilizzare il retro del foglio se necessario)

[] true (cond. ai limiti)
[0] true (cond. ai limiti)
[1] false (cond. ai limiti)
[0 3 4 2 1] true (combinazioni prop.)
[0 3 4 2 2] false (combinazioni prop.)

Esercizio 2 (punti 17)

Si consideri la seguente classe ***NumeroIllimitato*** che rappresenta numeri interi che possono crescere illimitatamente (cioe' senza essere limitati dal numero di bit della loro rappresentazione). Si supponga che la classe abbia la seguente specifica:

```
public class NumeroIllimitato {
//OVERVIEW: Rappresenta numeri interi, non negativi, in base 10 e la
cui dimensione è qualunque. Tipo Immutabile.
  //@ensures (* \result è il numero di cifre di this *)
  public int size();
  //@ensures (*costruisce NumeroIllimitato inizializzato all'intero
n*);
  public NumeroIllimitato(long n),
  //@ensures (* \result è un iteratore per scorrere le cifre dalla
meno significativa alla più significativa. Ogni cifra e' un Integer.
*)
  public Iterator cifre();

  //@ensures (* \result è la somma di this e n *)
  public NumeroIllimitato somma(NumeroIllimitato n)

  //@requires 1<=n && n<=9;
  //@ensures (* \result è ottenuto da this aggiungendo la cifra n in
base 10 nella posizione più significativa *);
  public addCifra(int n);
...piu' altri metodi che ignoriamo...
```

Parte (a).

Utilizzando solamente i metodi pubblici di NumeroIllimitato, si implementi, sotto l'ipotesi semplificativa che this e n abbiano lo stesso numero di cifre, il metodo **`somma:,`**

```
NumeroIllimitato res = new NumeroIllimitato(0);
int riporto =0; int c = 0;
for(Iterator i = this.cifre(), Iterator j=n.cifre(); i.hasNext();) {
        int c1= ((Integer) i.next().intValue();
        int c2= ((Integer) j.next().intValue();
        c = c1 + c2 + riporto;
        if (c>=10) {riporto =1; c = c-10;}
        res.addCifra(c);
}
if (riporto>0) res.addCifra(1);
return res;
```

Parte (b)

Si supponga che vi siano due implementazioni possibili per la classe NumeroIllimitato:

La classe NumeroIllimitatoString memorizza un numero in un oggetto di tipo String, utilizzando la consueta rappresentazione testuale in base 10 (la cifra 0 è memorizzata con il carattere "0", ecc.).

La classe NumeroIllimitatoBCD utilizza un array per memorizzare la sequenza di cifre. Ogni elemento dell'array è a sua volta un array di 4 bit, che codific una cifra decimale ([0,0,0.0] = 0, [0,0,0, 1] = 1, [0,0,1,0] = 2 , ... [1,0,0,1] = 9, mentre le codifiche corrispondenti ai numeri binari 1010, 1011, 1100, 1101, 1110, e 1111 non sono utilizzate).

Si mostri il rep e si tratteggi l'implementazione del metodo addCifra nei due casi.

NumeroIllimitatoString:

Il rep e':
private String num;

addCifra:
NumeroIllimitato nuovo = new NumeroIllimitatoString(0);
if (n>0 && n<=9) {
String cifra = String,valueOf(n); //converte n in stringa
nuovo.num = c+this.num;
return nuovo;
}

NumeroIllimitatoBCD;
Il rep e'
private boolean num[][];

addCifra:
NumeroIllimitato nuovo = new NumeroIllimitatoBCD(0);
if (n>0 && n<=9) {
boolean cifra[] = new boolean[4]:
"converte n nell'array cifra[];
nuovo.num = new boolean [this.length][4];
"copia in nuovo.num ogni riga di this.num";
nuovo.num[nuovo.num.length-1] = cifra;
return nuovo;
}

Parte (c) Si discutano brevemente vantaggi e svantaggi di ciascuna delle due implementazioni precedenti e si scriva l'invariante di rappresentazione come private invariant in JML per entrambi i casi:
L'implementazione con le stringhe e' piu' semplice da realizzare. L'implementazione con i BCD e' piu' macchinosa, ma, se i boolean sono effettivamente implementati con un singolo bit di memoria, puo' occupare molto meno spazio, prezioso per numeri di dimensioni illimitate.

RI per NumeroIllimitatoString:
num !=null && (\forall int i; 0<= i<num.size(); num.charAt[i] >= "0" && num.charAt[i]<="9");

Ri per NumeroIllimitatoBCD;
num != null && (\forall int i; 0<= i && i <num.size();num[i].size() == 4 &&
(!num[0] && !num[1] && !num[2] && !num[3] || NB: 0000
!num[0] && !num[1] && !num[2] && num[3] || NB: 0001
....
num[0] && !num[1] && !num[2] && num[3]) NB: 1001

Le ultime clausole formalizzano il fatto che ogni elemento di num[i] deve corrispondere a una rappresentazione BCD.

Parte (d)
Si realizzi per la classe NumeroIllimitatoString anche il generatore (cioe' la classe che implementa l'iteratore) per poter implementare il metodo ***Iterator cifre()***

In NumeroIllimitatoString si dichiara la seguente classe:

```
private class NumIllimitatoStringGen implements Iterator {
        private int pos;
        private NumIllimitatoString n;
        public NumIllimitatoStringGen(NumIllimitatoString t) {n=t; pos ==0;}
        public boolean hasNext() {pos <  n.size();}
        public Object next() throw NoSuchElementException{
                if (pos>= n.size()) throw new NoSuchElementException();
                return (n.charAt(pos).parseInt());
        }
}
```

Il metodo cifre() è implementato come:
return new NumeroIllimitatoStringGen(this);

NB: le soluzioni sono solo indicative e non corrispondono a codice Java immediatamente compilabile e eseguibile.

Parte (e)
Discutere brevemente, aiutandosi anche con diagrammi UML, come devono essere organizzate le classi in modo che:

1) gli utilizzatori della classe NumeroIllimitato non conoscano l'esistenza delle due classi NumeroIllimitatoString e NumeroIllimitatoBCD,
2) la scelta della rappresentazione possa essere realizzata da parte dei metodi produttori.

   Si possono proporre anche modifiche alle classi presentate, qualora necessarie per rispettare (1) e (2).

La classe NumeroIllimitato deve essere astratta e senza costruttori. L'implementazione e' realizzata dalle due classi eredi di NumeroIllimitato: NumeroIllimitatoString e NumeroIllimitatoBCD,
La classe NumeroIllimitato non fornisce costruttori, ma un metodo statico del tipo:

```
//result è un NumeroIllimitato pari a n
public static NumeroIllimitato costruisci(int n) {
}
```

In base ad esempio alle dimensioni di n, il metodo stabilisce se chiamare il costruttore di NumIllimitatoString o NumeroIllimitatoBCD.

L'implementazione dei produttori come somma dovra' includere anche la decisione del tipo effettivo dell'oggetto ritornato. Ad esempio, la NumeroIllimitatoString.somma() potrebbe restituire un NumeroIllimitatoString se anche il suo argomento è un NumeroIllimitatoString, ma altrimenti restituire un NumeroIllimitatoBCD.